Semantic-Driven Design

For Complex Energy Co-Simulation Platforms

Q. 1

"How do you teach a computer what you know, and make it reason about it?"

Cat

Domesticated

Persian

Loki

1

Meat

Cats are a kind of animals. → Loki is and animal.
Each kind of animal has a breed. → Loki is a perisan cat.
Animals are living things. → Loki is a living thing.
All living things eat food.
Foods are divided into Meat and Vegetable.
Cats eat meat. → Loki eats meat → Meat is a food.
Animals are wild or domesticated.
All domesticated animals have a name.
.
.
.

1. Knowledge Representation with Ontologies

1.1. What Is an Ontology?

"A formal, explicit specification of a shared conceptualization" [*1]

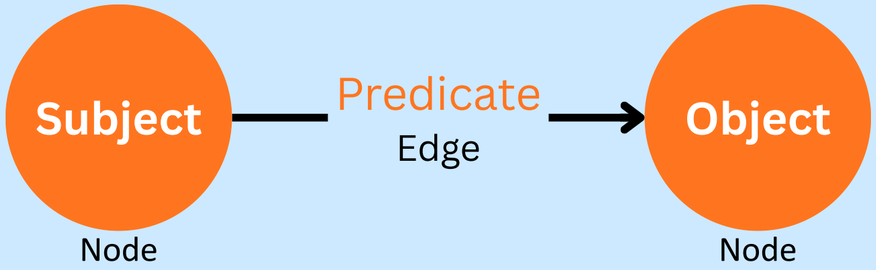
- ***Formal:*** *machine-readable, unambiguous*
- ***Explicit:*** *concepts and rules are clearly defined*
- ***Shared:*** *agreed upon by a community*
- ***Conceptualization:*** *an abstract model of some part of the world*

1 - R. Studer, V.R. Benjamins, D.Fensel, Knowledge engineering: Principles and methods, Data & Knowledge Engineering, Volume 25, Issues 1–2, 1998, Pages 161-197, ISSN 0169-023X, Link.

1.2. Graph-Based Knowledge Representation

Ontologies represent data as **Triples** (Subject-Predicate-Object).

- **Nodes**: Represent entities (Entities/Classes).
- **Edges**: Represent relationships (Properties).
- **Reasoning**: RDFS/OWL defines the logical rules.



1.3. Methodology: Ontology 101

A guide to creating your first ontology (Not & McGuinness [*2])

1. Scope

Determine domain, users, and competency questions.

2. Reuse

Import existing standards (SAREF, SMS, FMI, OEO)

3. Heirarchy

Define Classes and taxonomies (top-down/bottom-up)

4. Slots

Define properties (facets, constraints, cardinality)

2 - Noy, N.F. and McGuinness, D. (2001) Ontology Development 101: A Guide to Creating Your First Ontology. Stanford Knowledge Systems Laboratory. Link

1.4. Serialization Formats

- **Popular RDF-Based Formats:**
  - **Turtle (.ttl):** The most human-readable RDF format.
    - `:Reza a :Person .`
  - **RDF/XML (.owl, .rdf):** The original W3C standard serialization.
    - `<owl:NamedIndividual rdf:about=":Reza" rdf:type=":Person"/>`
  - **JSON-LD (.jsonld):** RDF expressed as JSON. Ideal for web APIs.
    - `{"@id": ":Reza", "@type": ":Person"}`
- **Other formats:**
  - **N-Triples (.nt):** Easy to parse, not human-friendly.
    - `<:Reza> <rdf:type> <:Person> .`
  - **RDFa:** Embeds RDF triples directly into HTML markup.
    - `<div about=":Reza" typeof="Person">Reza</div>`
  - **OBO Format (.obo):** Used heavily in bioinformatics (e.g., Gene Ontology).
    - `id: Reza  is_a: Person`

Selection depends on whether the goal is human curation, machine performance, or web interoperability.

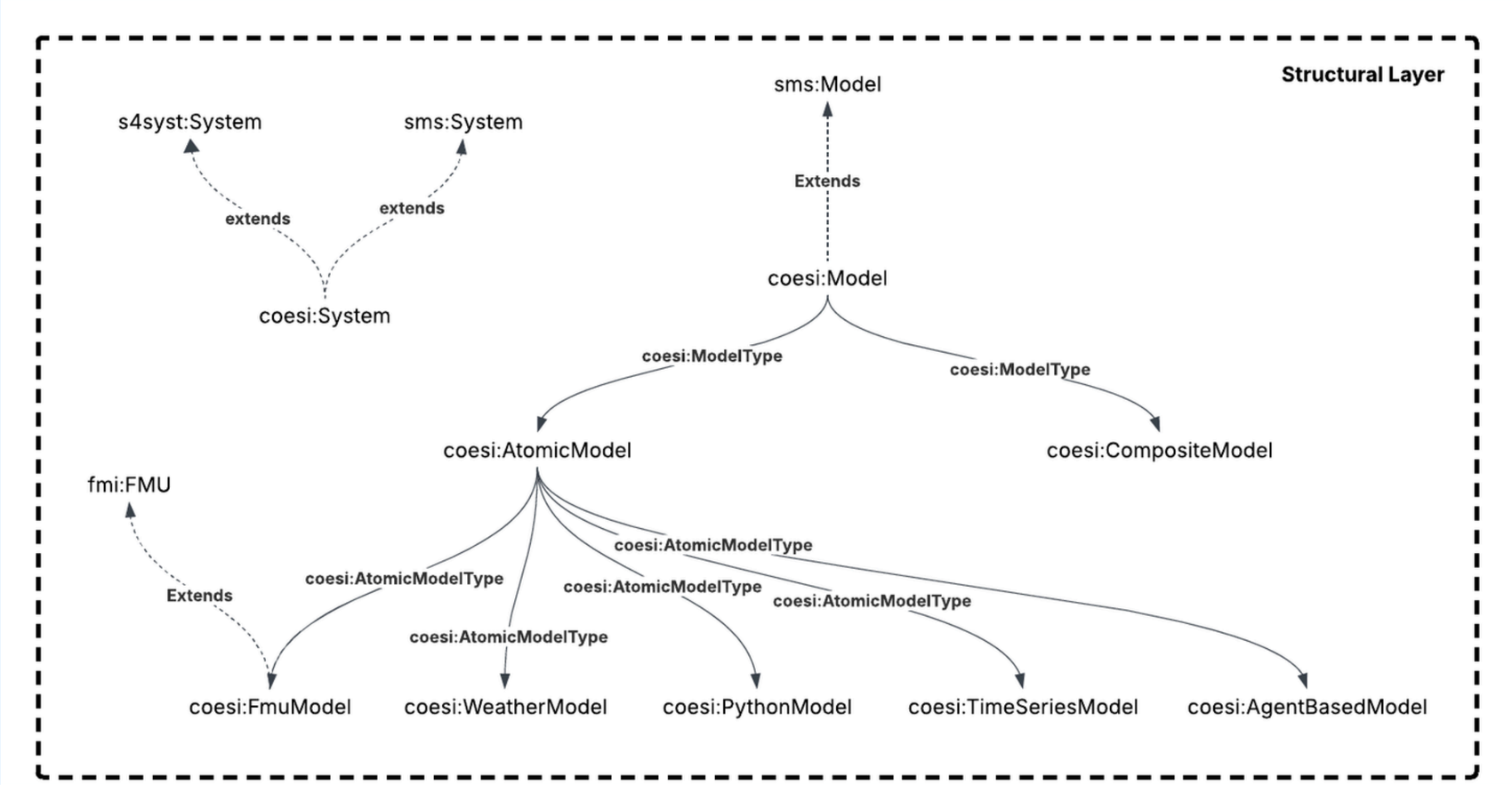
1.6. Tools & Technologies

- **Protégé:** The gold standard IDE for building OWL ontologies.
- **SPARQL:** SQL-like query language for retrieving data from graphs.
- **GraphDB / Apache Jena:** Robuts triplestores for hosting knowledge bases.[*3]

3 - Neo4j is another option, but it naturally build for being a graph database not a native Triplestore. So we need to install n10s plugin, to be able to import RDF data, apply SHACL validations, etc.

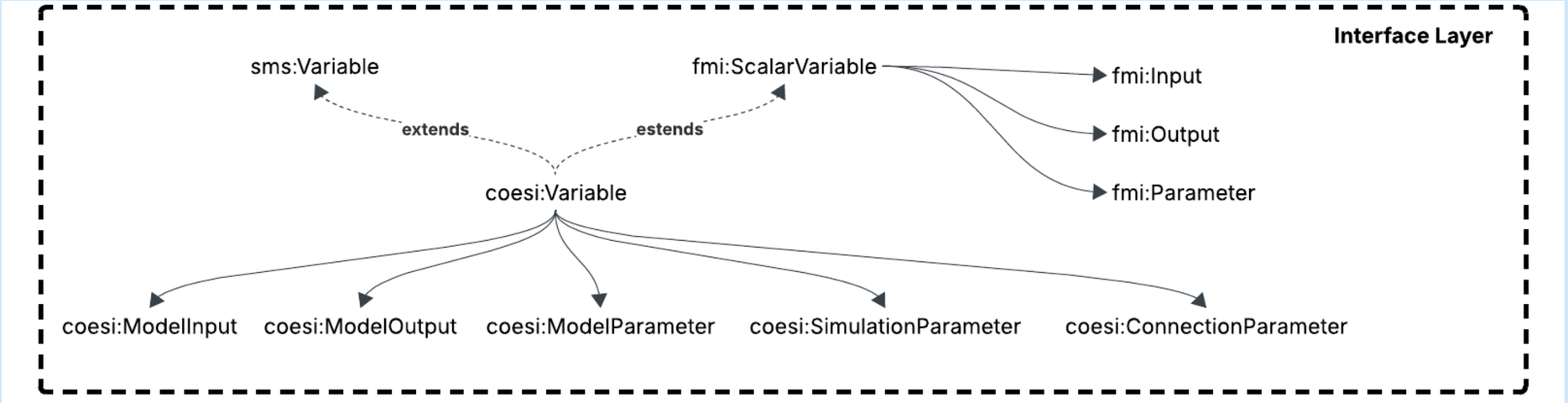
2. COESI Ontology

2.1. Current Structural Layer*4

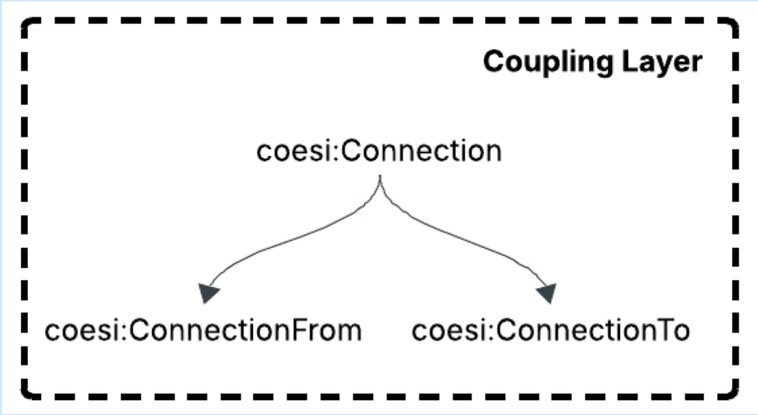


**4 - I am referring to this as the 'current' version because it represents our validated and stable baseline. While we have more comprehensive iterations in development, they are still undergoing the formal approval process and are not yet ready for deployment.*

2.2. Current Interface Layer



2.3. Current Coupling Layer



Q. 2

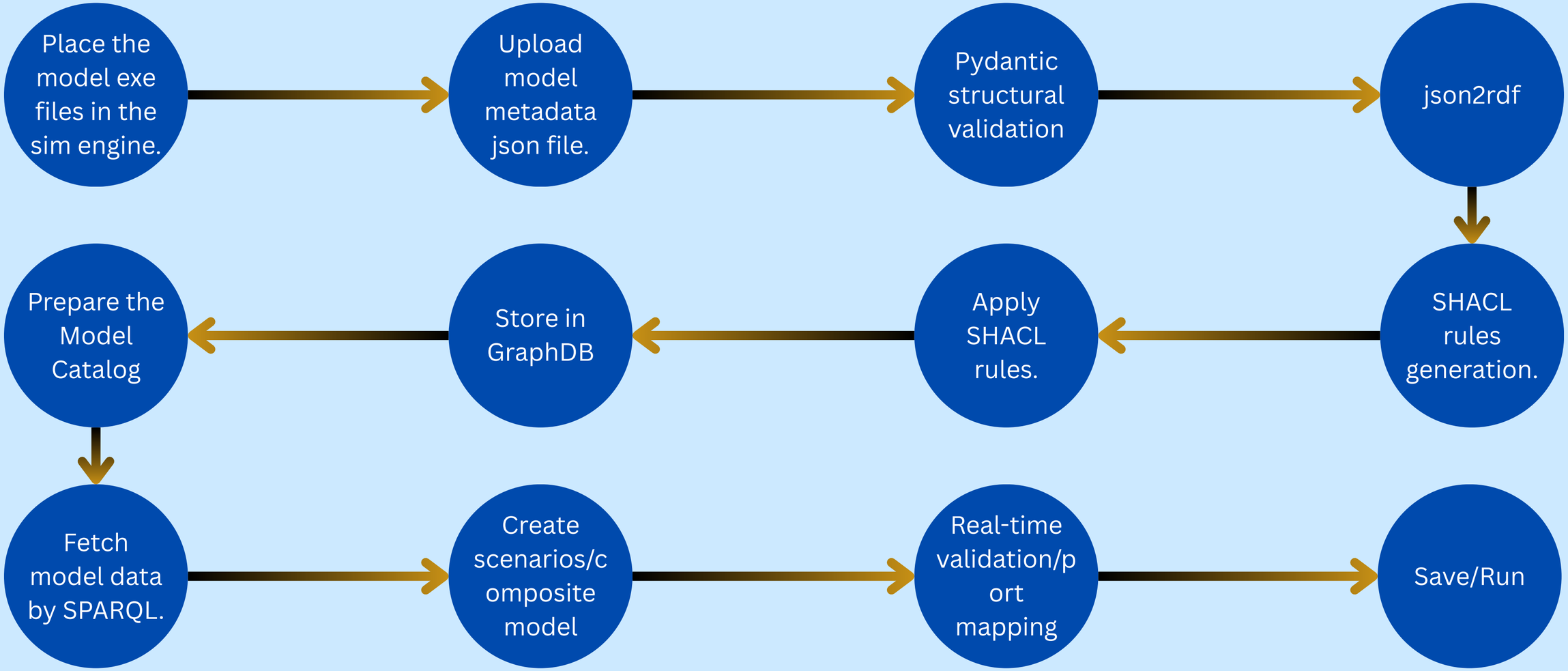
To achieve seamless interoperability, what additional semantic layers or vocabularies do you feel are missing from our current roadmap?

3. The Validation Engine

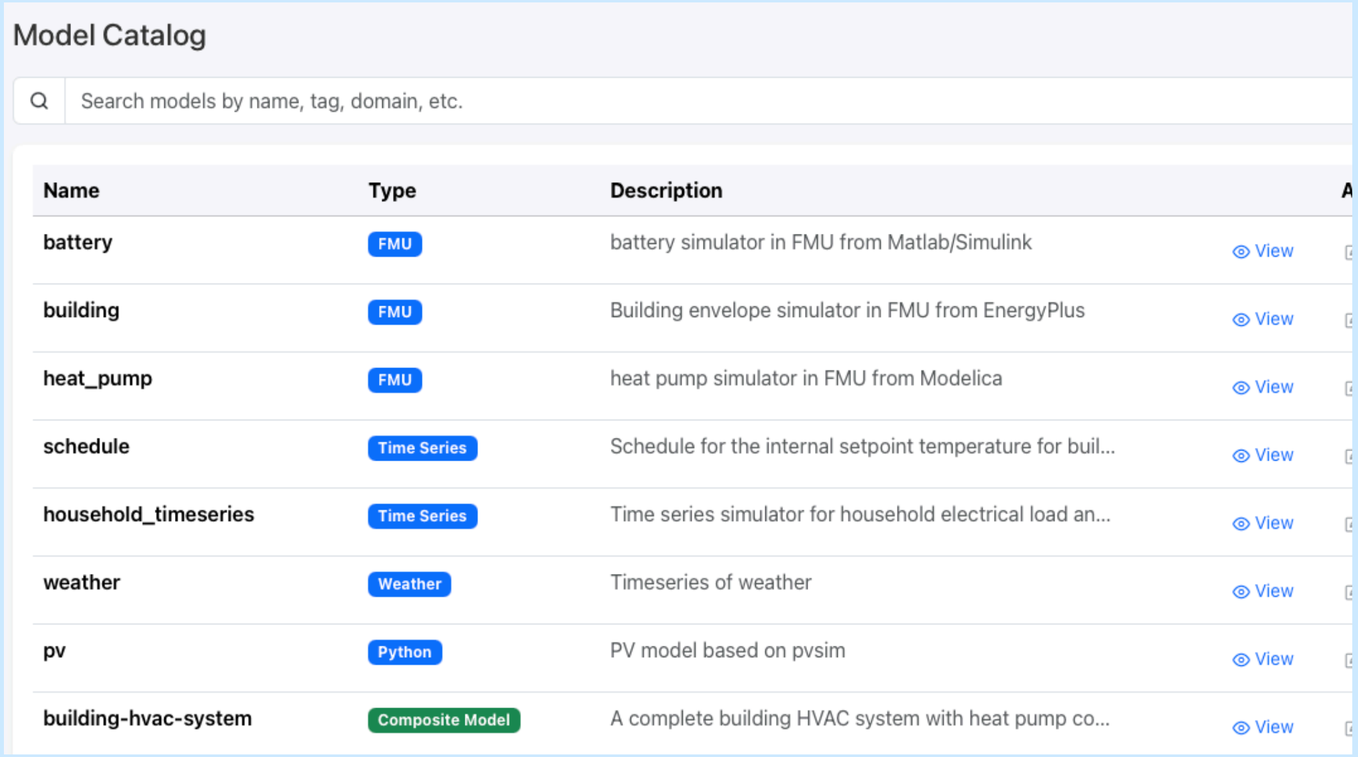
Our validation engine verifies every connection during scenario configuration. Key checks include:

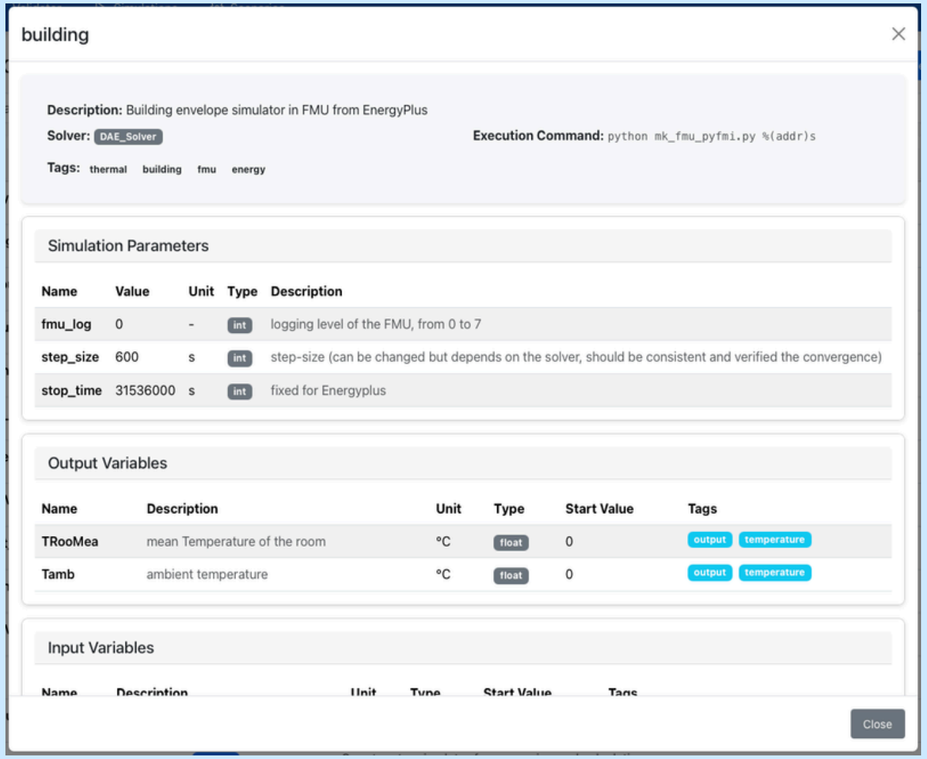
1. **Semantic Compatibility**: Prevents cross-commodity errors (e.g., linking Electricity to Heat) via tag matching.
2. **Directional Logic**: Enforces strict output-to-input causality across all ports.
3. **Unit Of Measurement Consistency**: Validates units of measures to ensure mathematical accuracy.
4. **Datatype Alignment**: Ensures datatype synchronization.

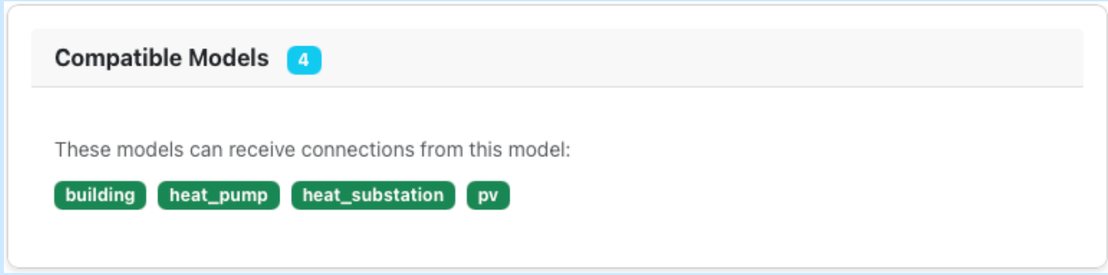
4. Workflow


Place the model exe files in the sim engine.
Upload model metadata json file.
Pydantic structural validation
json2rdf
SHACL rules generation.
Apply SHACL rules.
Store in GraphDB
Prepare the Model Catalog
Fetch model data by SPARQL.
Create scenarios/composite model
Real-time validation/port mapping
Save/Run

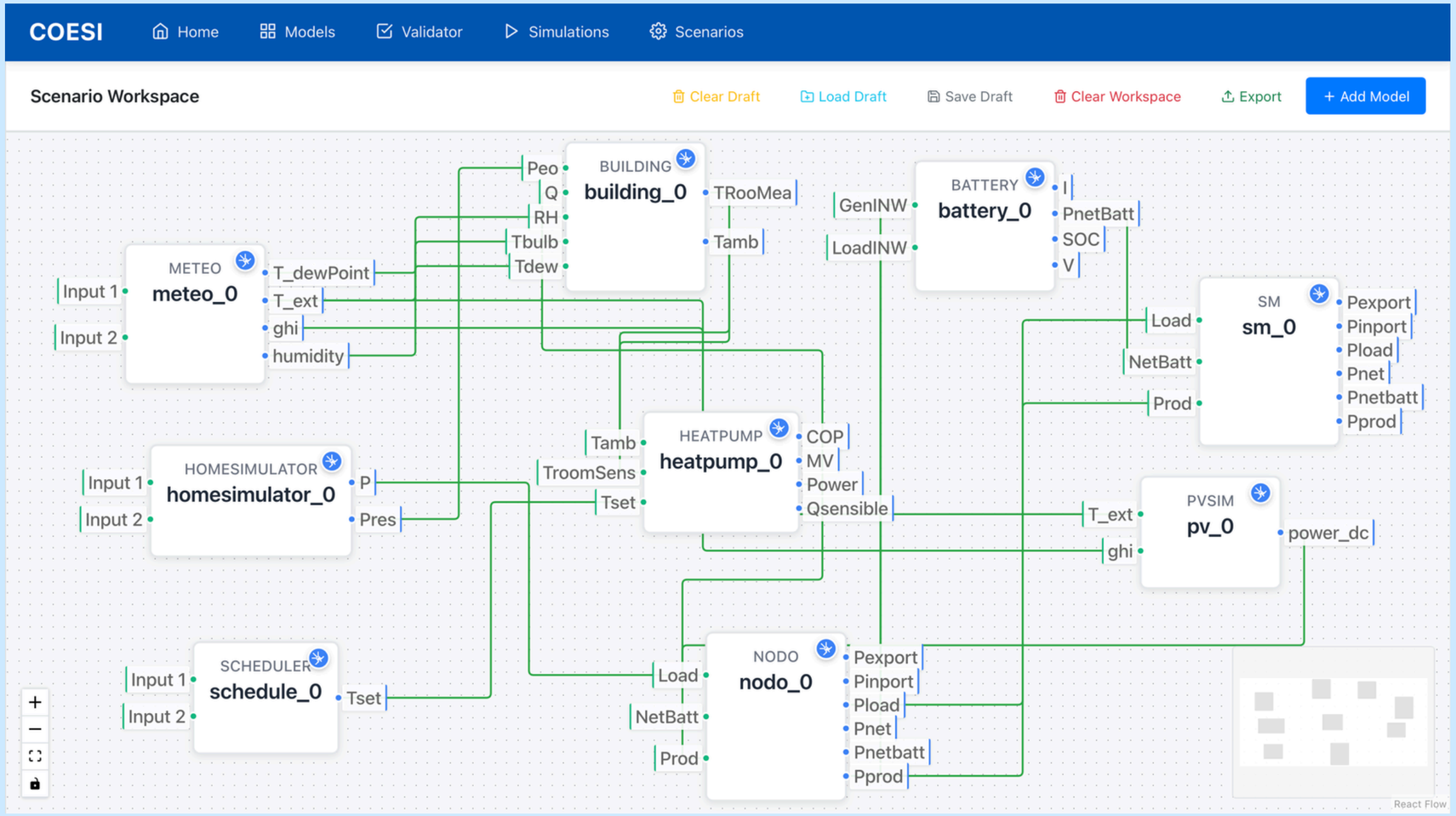
5. The Model Catalog







6. The Dashboard



Success

Connection created successfully!
Source: weather0.T_ext
Target: pv0.T_ext

Error

Invalid connection: gridNode1.P_ac → battery1.P_dc
Reasons:
• Electrical characteristics mismatch: source is 'AC' while target is 'DC'
• Direct AC → DC connection is not allowed without a conversion component

Error

Invalid connection: pv1.P_el → building1.Q_heat
Reasons:
• Commodity mismatch: 'P_el' has commodity 's4ener:ElectricPower' but 'Q_heat' has commodity 's4ener:HeatThermalPower'
• No valid energy-carrier mapping exists for a direct connection

7. Conclusion

7.1. Core Platform Capabilities

- Semantic Error Prevention
- Automated Configuration
- Scalable Microservices Architecture
- Model Reusability & Discovery
- Foundational Layer for AI

7.2. Limitations

- **Monolithic Architecture of the Simulation Engine**: Any minor model update currently requires a full rebuild of the entire platform.
- **Limited Physical Awareness**: The ontology lacks deep integration of lower-level physical and orchestration constraints.
- **Complex Deployment**: The core platform is oversized, leading to a slow and time-consuming deployment process.
- **Service Dependency**: Although the platform is microservice-based, but there's still room for improvement and the Model Catalog needs to be decoupled into a standalone service with an API.
- **Orchestration Lock-in**: Support is currently limited to mosaik; integration with HELICS is still required.

7.3. Future Works

- **Simulation-as-a-Service (SIMaaS):** Transitioning to a micro-provider architecture where every simulation model operates as an independent, scalable service.
- **Ontology Refinement:** Finalizing and deploying the comprehensive version of our internal ontology to capture more complex domain knowledge.
- **Enhanced Reasoning Core:** Integrating the HermiT reasoner to improve performance and support more complex logical classifications.
- **Orchestrator-Agnostic Design:** Enabling multi-platform support so users can switch seamlessly between mosaik and HELICS depending on their needs.
- **Independent Model Catalog:** Decoupling the catalog into a standalone, server-deployed service for better accessibility and model sharing.
- **Advanced Data Distribution:** Integrating message brokers and InfluxDB for high-performance time-series data handling.
- **Full-Stack Monitoring:** Implementing the LGTM stack (Loki, Grafana, Promtail) for real-time system observability and performance tracking.
- **Metadata Enrichment Service:** Developing a dedicated service to automatically extract and enrich model metadata, ensuring deeper semantic alignment within the catalog. (Automated for FMUs/ CSVs)

Thanks!